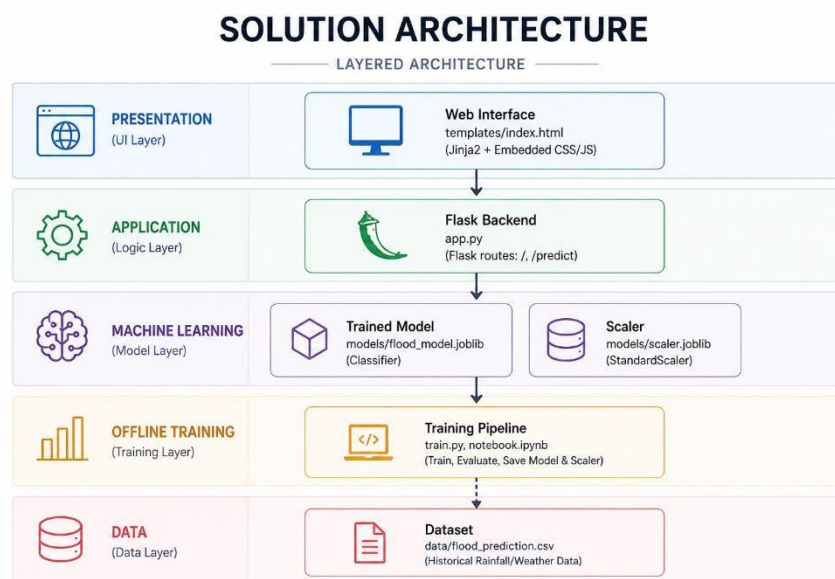


Project Title	Rising Waters: Machine Learning Based Flood Prediction System		
Developer	N. Joshnavi devi	Team ID	[Team ID]
Date	26 June	Phase / Document	Project Design Phase

10. Solution Architecture

The architecture follows a simple layered pattern rather than a microservice or event-driven design, which is appropriate given the project's single-user, single-process demonstrative scope. Each layer below has one clear responsibility and communicates with the layer above or below it through a narrow interface (form data in, rendered HTML out; DataFrame in, prediction out), which keeps the system easy to reason about end-to-end.



10.1 Layered Architecture

Layer	Component	Responsibility
Presentation	templates/index.html (Jinja2 + embedded CSS/JS)	Collects user input, renders prediction, insights, and protocols; provides Auto-Fill demo generator.
Application	app.py (Flask routes: /, /predict)	Parses form input, orchestrates scaling and prediction, applies severity/insight rules, returns rendered response.
Machine Learning	models/flood_model.joblib, models/scaler.joblib	Pre-trained classifier and feature scaler loaded once at application startup.
Offline Training	train.py, notebook.ipynb	Loads and cleans dataset, trains and compares models, evaluates, and serializes the best-performing model and scaler.

Data	data/flood_prediction.csv	Historical rainfall/weather records with binary flood-risk label used for training.
------	---------------------------	---

A useful way to read this table is top-to-bottom as a request's lifecycle: a request enters at the Presentation layer, is handed to the Application layer for orchestration, borrows a fitted estimator from the Machine Learning layer, and never touches the Offline Training or raw Data layers at all during serving — those only run ahead of time, whenever the model needs to be (re)trained.